

Programmierpraktikum: Federated Machine Learning (FML)

Target audience: B.Sc. Computer Science & Business Informatics + others

Goal: Development of a Federated Machine Learning application

Scope: 6 ECTS credits, portfolio examination

Learning outcomes

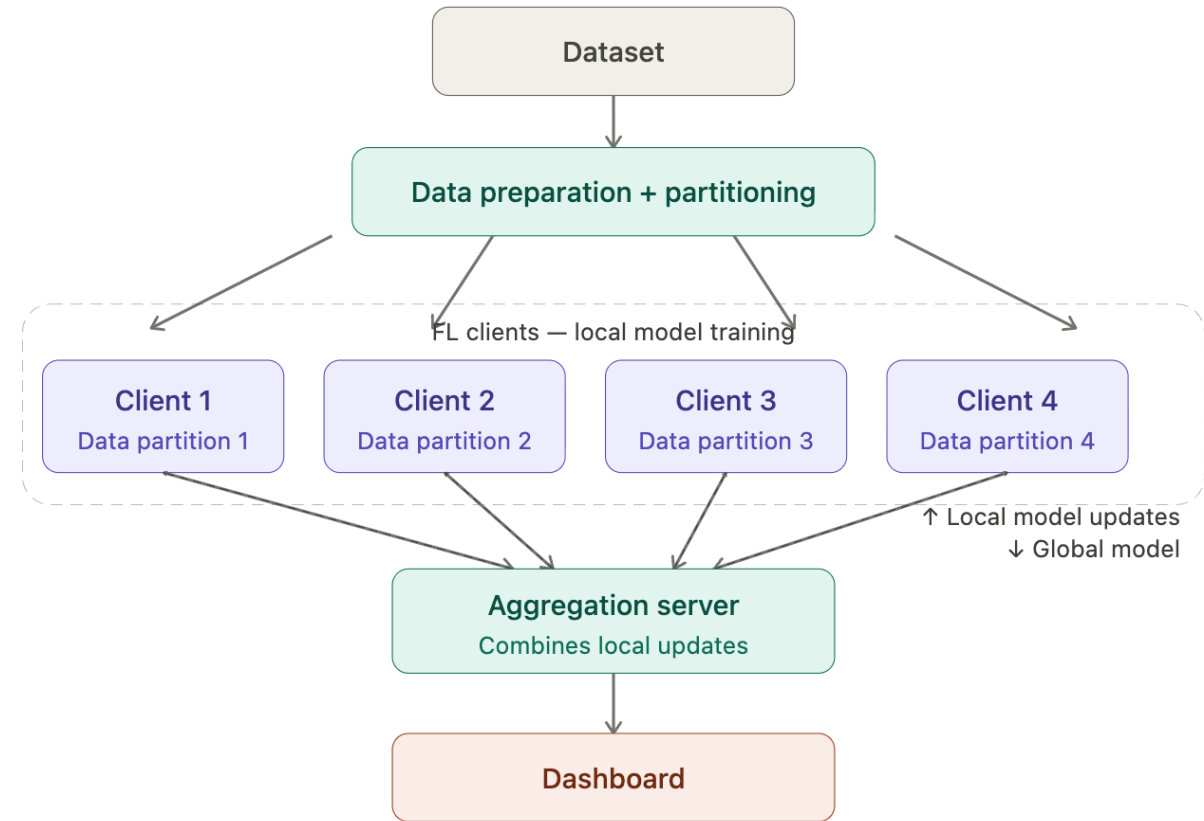
- Practical application of federated learning technologies for specific applications
- Team-oriented project work
- DevOps use of cloud-native tooling for deployment

See module description for details on module structure and examination

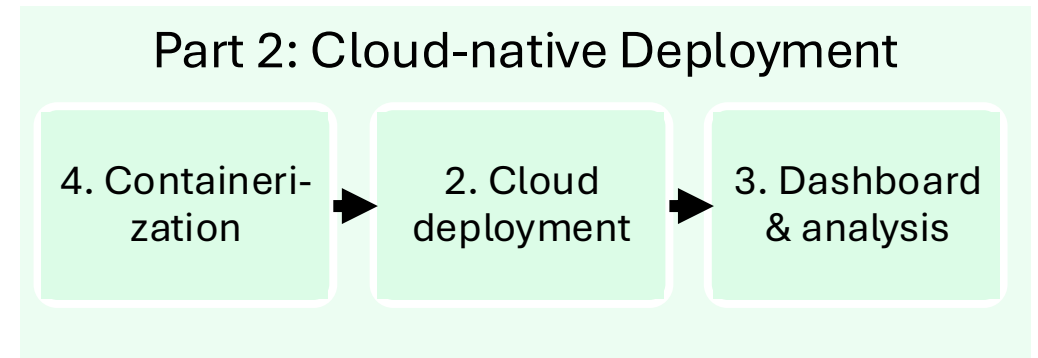
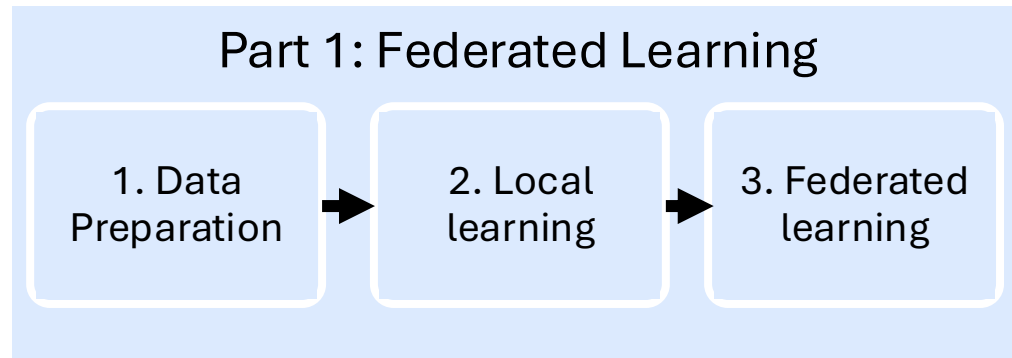
Task: You design, implement, and evaluate a federated learning system for a specific application domain.

While the domains differ, every team builds the same core architecture:

- A **data preparation pipeline** loads, preprocesses, and partitions the dataset into separate subsets, one per simulated client.
- A **set of FL clients** each train a local model on their own data partition and send model updates to a
- **central aggregation server**, which combines the local models into a global model and distributes it back for the next training round.



Project development in two parts



→ Same architecture and same procedure, but different topics!

Topic A – Sentiment Analysis

Machine Learning.

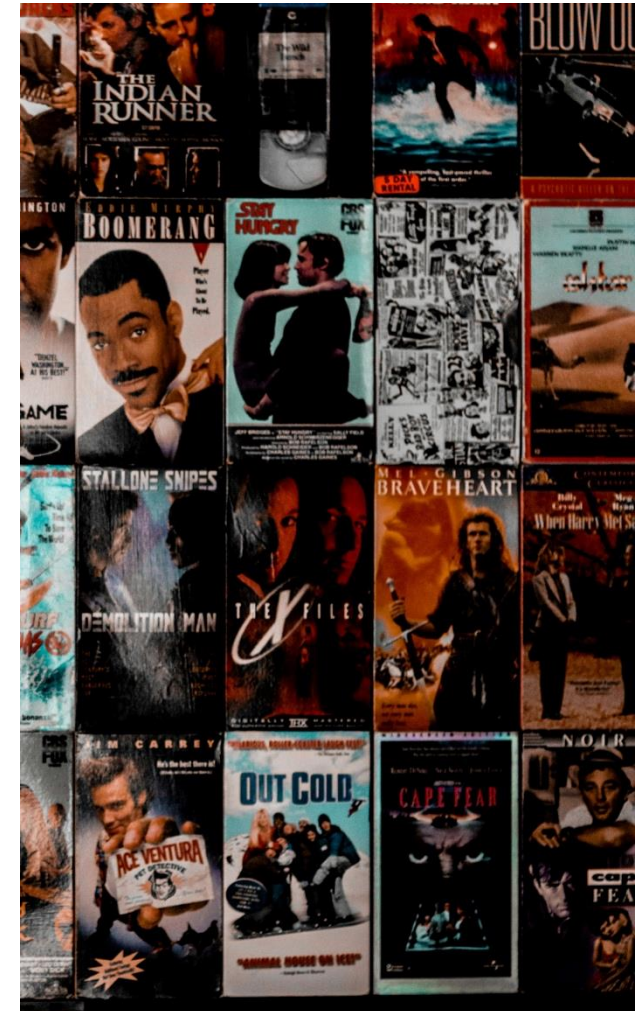
- *Goal:* build a model that classifies customer reviews as positive or negative.
- *Input:* text reviews.
- *Task:* binary text classification.
- *Output:* a sentiment label (positive / negative).

Federated learning.

Competing **streaming platforms** collect customer reviews. Each client represents one platform; clients jointly train a sentiment classifier while each platform's feedback data stays in-house.

Dataset: IMDB Large Movie Review Dataset

- <https://huggingface.co/datasets/stanfordnlp/imdb>
- <https://ai.stanford.edu/~amaas/data/sentiment/>



Topics Overview

	Topic A	Topic B	Topic C
<i>Domain</i>	Sentiment	Activity recognition	Handwriting
<i>Dataset</i>	IMDB reviews	UCI HAR	FEMNIST
<i>Data modality</i>	Text	Tabular (sensor features)	Grayscale images (28×28)
<i>Task</i>	Binary classification	6-class classification	62-class classification
<i>FL scenario</i>	Platforms sharing review classifiers	Users sharing fitness models	Schools sharing handwriting models

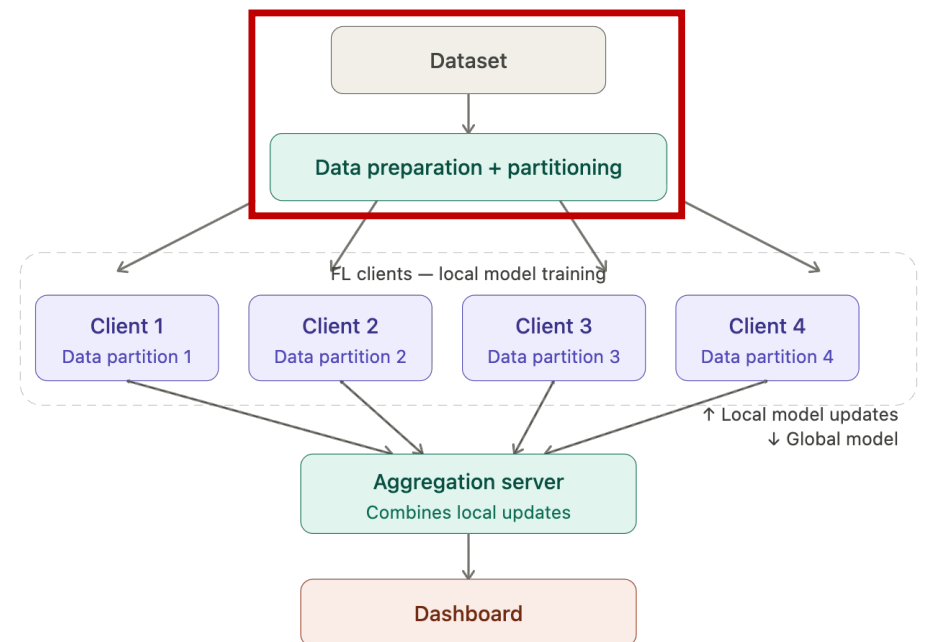
Step 1: Data preparation

Download the dataset, explore and preprocess it, partition it into four client subsets.

At this stage, a simple random split is fine, the focus is on getting clean, loadable data for each client.

Create a preprocessing tool that prepares the dataset for learning.

Technologies: Python and Python libraries like pandas, NumPy, HuggingFace Datasets



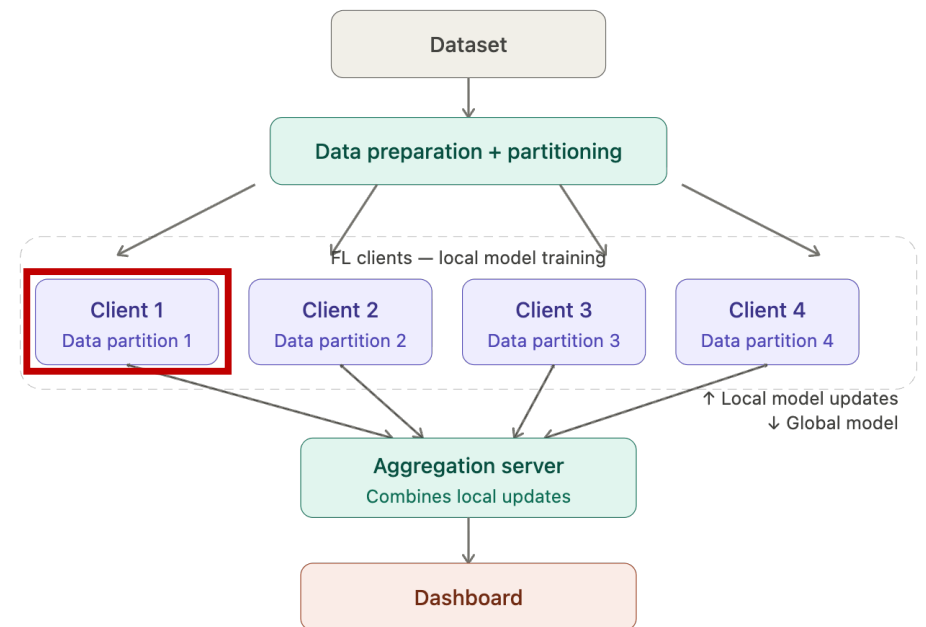
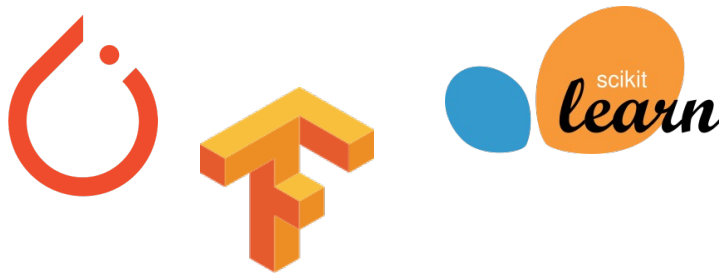
Step 2: Local model training

Build a single ML model and train it on one data partition.

Verify that the model learns: loss decreases, accuracy improves.

This is pure ML, no federation yet.

ML frameworks for model definition, training, and evaluation: PyTorch, TensorFlow, scikit-learn



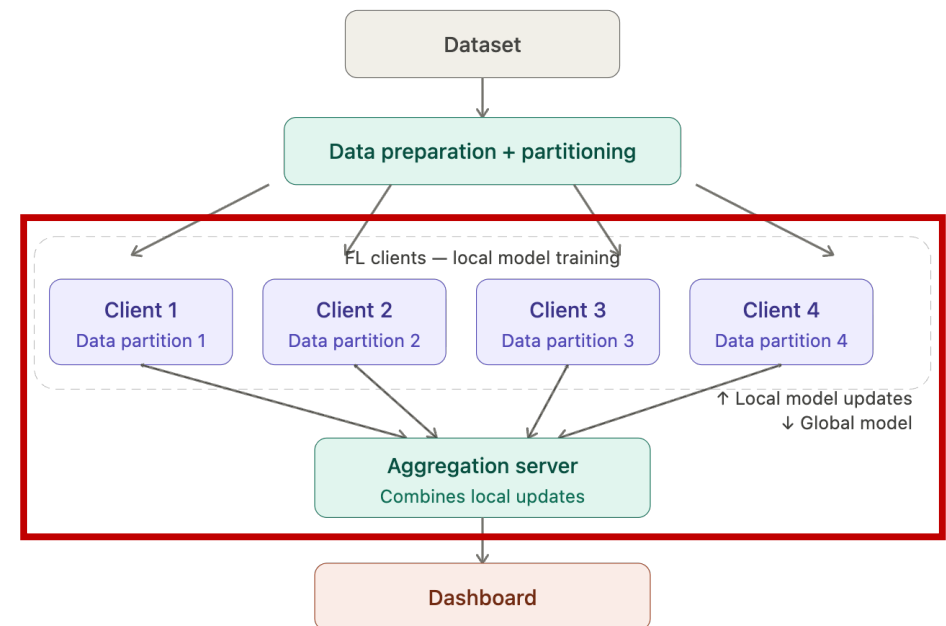
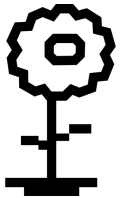
Step 3: Federated training

Connect four clients to an aggregation server.

Each client trains locally and sends model updates; the server aggregates and distributes the global model back.

Run multiple rounds and verify that the global model improves.

FL frameworks for client-server orchestration, aggregation, and communication: Flower, NVIDIA FLARE, TensorFlow Federated, OpenFL



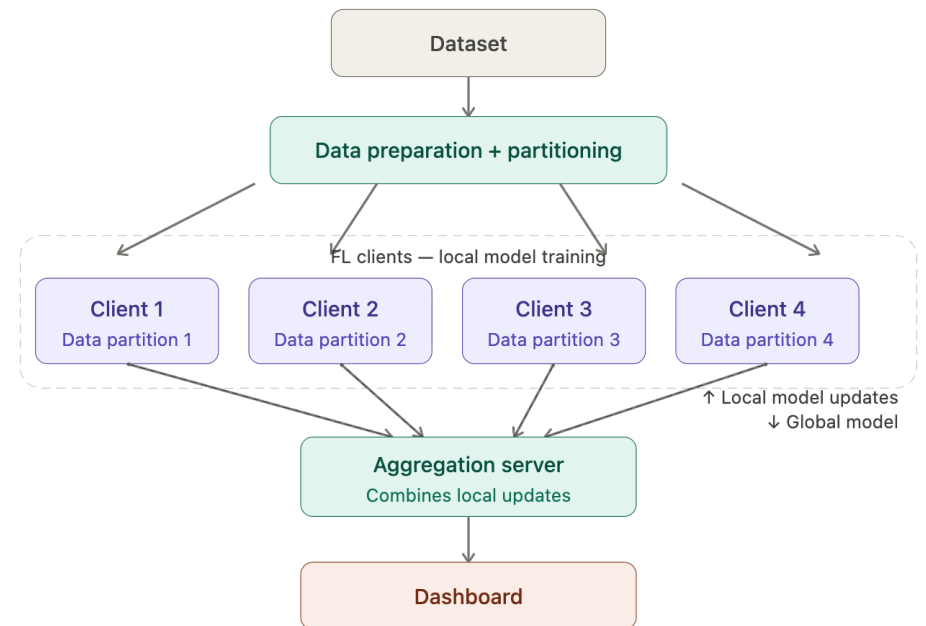
Step 4: Containerization

Package each component (server, four clients, dashboard) into its own container.

Orchestrate them so the full system starts with a single command.

Test locally first.

Containerization for packaging components into portable, reproducible units: Docker, Docker Compose



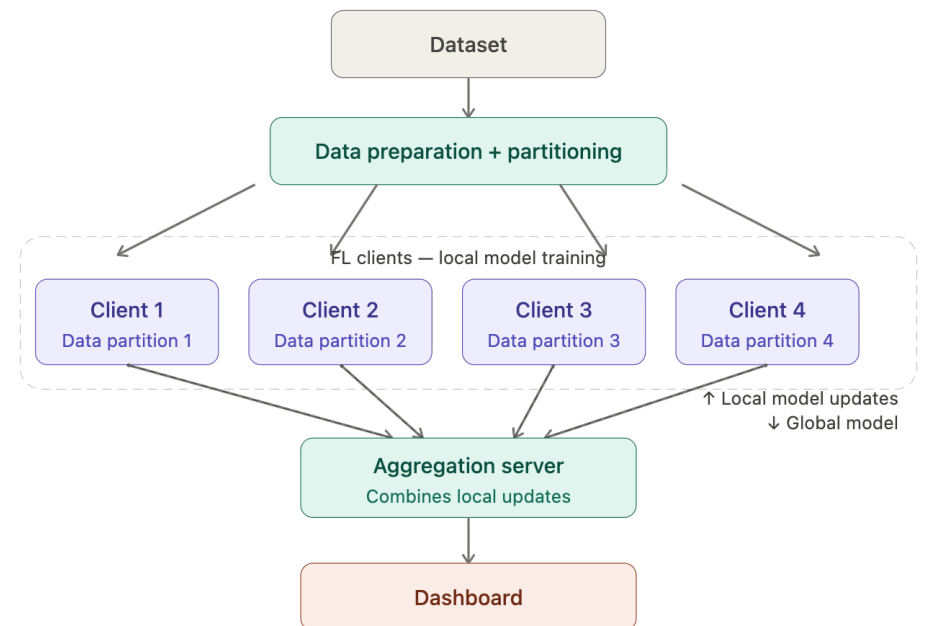
Step 5: Cloud deployment

Deploy the containerized system on cloud infrastructure.

Ensure clients and server communicate across the network.

Verify that training runs successfully in the cloud environment.

Cloud infrastructure for deploying and running the distributed system Examples: Google Cloud Platform, AWS, Azure

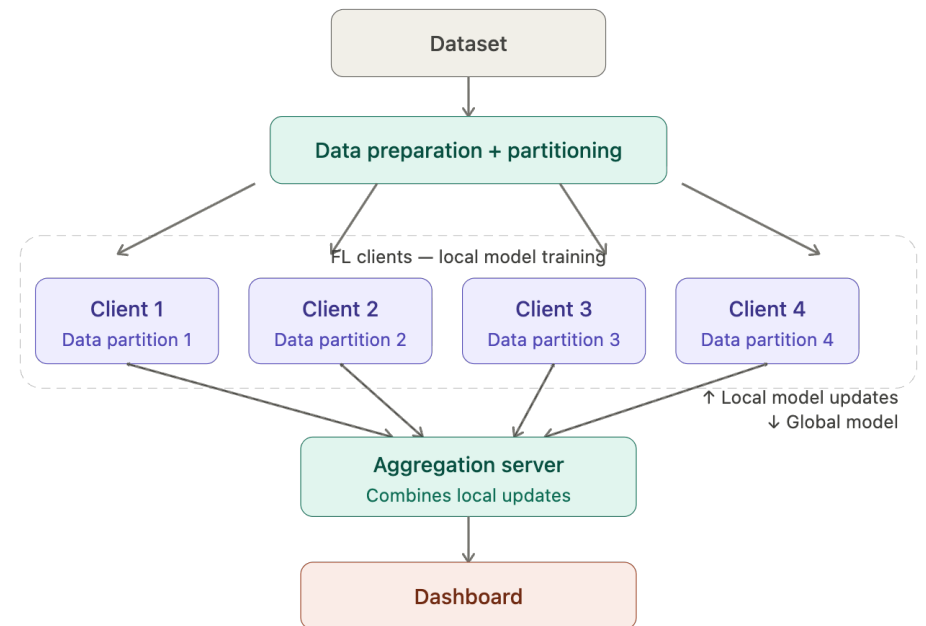


Step 6: Dashboard and analysis

Build an interactive dashboard that loads the global model and provides inference (enter a character / review / sensor reading, see the prediction).

Add visualizations of training metrics (convergence curves, per-client performance).

Dashboard and visualization tools for building interactive web interfaces for inference and visualization



Choose your tech stack

1. Data Preparation
2. Local Machine Learning
3. Federated Machine Learning
4. Containerization
5. Cloud Deployment
6. Dashboard and Analysis

Python + lib



PyTorch



Flower



Docker



Google



??

Disclaimer: We do not know all details of all frameworks!!!